PROJECT FOR QNX

Prerequisite for any project.

1. Must make use of QNX message API

2. client server architecture must be followed.

3. There should be at least one IOV transfer from client to server or vice- versa.

4. Payload between client and server shall be at least 64 KB for operations like checksum, encryption etc.

5. The message definition shall be in a common header file.

6. The source code shall be well commented, and variable names shall be meaningful. 7. The client code shall be implemented in such a way that it waits for the server and timeout if not able to connect for more than 10 Seconds.

8. All APIs return value has to be checked and either operation has to be retried, or appropriate message shall be displayed.

9. The client shall connect to the server using the name open API only.

10. The server shall send the processed data or response asynchronously using an event.

Project Titles:

1. Concurrent File Encryption Service

The client sends a 64KB+ data block to the server. The server creates a pool of threads to encrypt different segments of the data concurrently. Mutexes protect shared resources (like the output buffer), and condition variables signal when all threads are done. The server sends the encrypted data back asynchronously.

2. Real-Time Sensor Data Aggregator with Threaded Processing

The client simulates a sensor, sending 64KB+ of readings. The server uses a thread pool to process (average/filter) the data in parallel. Mutexes and condition variables synchronize the aggregation and response.

3. Image Processing Service with Threaded Filters

The client sends a 64KB+ BMP image chunk. The server splits the image into segments and processes each segment in a separate thread (e.g., grayscale conversion). Mutexes and condition variables synchronize the merging of processed segments.

4. Shared Memory Data Logger with Timed Snapshots

The client collects or generates a large dataset (64KB+), then writes it to a shared memory segment. The server maps the same shared memory and, using a POSIX timer (timer_create), periodically takes a snapshot of the data, computes a checksum, and sends the result back to the client asynchronously via an event. Mutexes and condition variables ensure safe access to the shared memory region.

5. Real-Time Notification System with Shared Memory and Timers

Multiple clients write notification messages (each 64KB+) into a shared memory queue. The server monitors the shared memory and, using a POSIX timer, checks for new messages at regular intervals. When new data is detected, the server processes the message (e.g., logs it, sends an acknowledgment, or performs an operation) and notifies the client asynchronously via an event. Mutexes and condition variables manage concurrent access to the shared memory queue.

6. High-Throughput Packet De-Duplication and Checksum Engine

The client simulates a high-speed network interface, dispatching network frame buffers (≥64 KB) to the server via scatter-gather IOVs (MsgSendv()). The server splits the incoming buffer across worker threads using a producer-consumer model protected by POSIX mutexes and condition variables. Each worker thread calculates an Adler-32 or CRC-32 checksum on assigned sub-blocks to detect redundant frames. Once complete, the server notifies the client asynchronously using a QNX pulse/event (MsgDeliverEvent()), signalling the client to fetch or read back the verified verification bitmask.

7. Asynchronous Audio Buffer Equalizer and Peak Detection Service

The client captures raw PCM audio samples (≥64 KB) and delivers them to the server via an IOV array. The server assigns chunks of the audio frame to a worker thread pool to compute peak amplitude, normalize dynamic range, and apply software gain scaling in parallel. Mutexes guard the shared output structure, and condition variables track task completion. Upon completing the audio normalization, the server triggers an asynchronous POSIX signal or pulse event back to the client, delivering status codes and peak statistics.

8. Real-Time Telemetry Data Compression Engine

The client collects bulk automotive/industrial diagnostic logs (≥64 KB) and transmits them using a header-plus-payload IOV structure. The server divides the uncompressed payload across worker threads, each executing a Run-Length Encoding (RLE) or Byte-Pair algorithm on its segment. Mutexes serialize writes to the shared compressed output buffer, and condition variables orchestrate chunk synchronization. The server alerts the client asynchronously through an event containing the compressed payload size and descriptor offset.

9. Parallel Matrix Transformation and FFT Acceleration Server

The client sends a large 2D sensor grid dataset (≥64 KB) using multi-part IOV vectors. The server delegates partitioned row/column blocks to worker threads executing a 1D/2D Fast Fourier Transform (FFT) or matrix kernel convolution. Worker threads coordinate through POSIX barrier synchronization and condition variables to eliminate race conditions during intermediate transform stages. When the frequency-domain conversion completes, the server delivers an asynchronous event to the client to trigger result retrieval.

10. Real-Time Multi-Core Sorting and Median Filter Service

The client continuously offloads unsorted floating-point time-series metrics (≥64 KB) to the server. The server implements a parallel Merge-Sort algorithm across multiple worker threads, using mutexes and condition variables to control the recursive merging phases across partitioned memory. In addition, the server computes rolling median filters to

eliminate sensor outliers. On completion, the server sends a QNX pulse event back to the client with the calculated median and memory-offset descriptor.

11. Shared Memory Circular Flight Recorder with Watchdog Timer

The client writes continuous avionics flight telemetries (≥64 KB) directly into a named shared memory block (shm_open) managed by a custom circular ring buffer. A dedicated POSIX periodic timer (timer_create) wakes the server to scan buffer segments, run an anomaly-detection algorithm, and evaluate rate-of-change limits on critical parameters. Mutexes and condition variables handle read/write head collision avoidance. If an anomaly is identified, the server immediately dispatches an asynchronous event to the client with critical error flags.

12. Parallel Cryptographic Hash & Merkle Tree Verification Engine

The client splits a continuous firmware update binary (≥64 KB) across IOV fragments and sends it to the server. The server spawns worker threads to compute intermediate SHA-256/MD5 leaf hashes for each 4 KB sub-block concurrently. Worker threads synchronize through condition variables and mutex to construct a hierarchical Merkle Tree root. The server then validates the root hash against a predefined key. Finally, the server signals the client asynchronously via an event containing root integrity flags and block corruption indices.

13. High-Priority Sensor Alarm Dispatcher & Event Logger

The client bundles simulated multi-channel industrial sensor streams (≥64 KB) into an IOV array consisting of a metadata header and a bulk sample buffer. The server reads the packet and distributes sections of the data across a pool of worker threads to scan for out-of-band limit breaches (e.g., thermal thresholds, pressure spikes). Mutexes protect the shared alarm summary buffer, while condition variables signal the parent server thread when the scan completes. Once evaluated, the server alerts the client asynchronously using an event containing the count and location of critical alarms.

14. Asynchronous Multi-Channel Audio Down sampling Service

The client streams uncompressed 48 kHz PCM audio data (≥64 KB) using multi-part IOV structures. The server distributes independent interleaved audio channels across worker threads to perform integer decimation/downsampling (e.g., 48 kHz to 16 kHz) and bit-depth scaling concurrently. Shared write buffers are synchronized via mutexes, and condition variables orchestrate completion status. The server signals the client asynchronously via a QNX event/pulse with memory pointers to the reduced audio stream.

15. Batch String Tokenizer & Word Frequency Counter

The client transmits a raw text corpus or server log file (≥64 KB) to the server via scatter-gather IOVs (MsgSendv()). The server partitions the character buffer into line-aligned chunks and tasks worker threads with tokenizing strings, stripping whitespace, and generating local word counts. Mutexes guard the shared global frequency dictionary, while condition variables indicate when all sub-blocks have been reduced. Upon completion, the server delivers an event asynchronously notifying the client that top keyword statistics are ready for collection.

16. Network Packet Header Stripper & Payload Sanitizer

The client sends encapsulated network traffic frames (≥64 KB) where protocol headers (e.g., custom framing or legacy field tags) must be inspected and stripped. The server deploys worker threads to validate individual frames, sanitize sensitive payload segments, and zero out discarded headers in parallel. Mutexes synchronize status tracking for corrupted frames, and condition variables synchronize thread joins. The server fires an asynchronous event containing a validation bitmask back to the client once all frames are sanitized.

17. Parallel Bit-Level Matrix Transposition Engine

The client passes a large 2D bitmap or binary state array (≥64 KB) across to the server. The server assigns non-overlapping rows and columns to a worker thread pool to perform block-wise matrix transposition concurrently. Condition variables and mutexes track progress across worker segments to prevent access violations. When transposition finishes, the server sends an asynchronous event containing completion flags and throughput metrics back to the client.

18. Log File Keyword Search & Error Counter

The client sends a large system log file (≥64 KB) to the server using an IOV transfer. The server splits the text into equal sections and assigns each section to a worker thread. Each thread counts occurrences of key error words (like FAIL, TIMEOUT, ERROR). Mutexes protect the total count variables, and condition variables signal when all threads are finished. Once done, the server sends an asynchronous event back to the client with the total error counts.

19. CSV Sensor Data Validator & Range Checker

The client sends a bulk CSV text buffer (≥64 KB) containing rows of sensor readings (ID, temperature, voltage). The server splits the rows across a thread pool. Each thread parses numbers, checks if any values exceed predefined safe limits, and flags corrupted rows. A mutex protects the shared list of invalid records, and condition variables synchronize thread completion. The server alerts the client asynchronously via an event with the total number of flagged readings.

20. Large Array Min/Max & Statistics Finder

The client generates an array of floating-point numbers (≥64 KB) and delivers it to the server via IOVs. The server assigns chunks of the array to worker threads to simultaneously find the minimum, maximum, and running sum. Mutexes protect the shared global min/max variables, and condition variables coordinate the final result. The server notifies the client asynchronously using an event containing the overall minimum, maximum, and average values.

21. Batch 1D Signal Moving Average Filter

The client sends raw 1D sampled sensor signals (≥64 KB) to the server. The server divides the raw array among worker threads to apply a simple 5-point moving average filter to smooth out noise. Worker threads write the smoothed values into a shared output buffer using mutexes to prevent race conditions at block edges, and condition variables signal completion. The server then triggers an asynchronous event notifying the client that filtered data is ready for retrieval.